COMP132: Advanced Programming

Programming Project Report

Food Chain Through Time: Simulation Design and Development

<Baran Ecertemel, 0087279>

<Fall 2025>

Part 1: General Demo Information

1)Introduction: This guide outlines the core features of the "Food Chain Through Time" application. It covers system configuration, GUI functionality, and the underlying game logic. You will also find details on the application's logging infrastructure and data persistence methods.

2)Configuration Files (.txt): The application uses external text files for its setup. This approach allows you to tweak the game environment instantly, without the need to recompile the code every time you make a change. The ecosystem's hierarchy is established using files like past_animals.txt, present_animals.txt, and future_animals.txt. The Reader class processes these files to dynamically generate the correct Animal objects for the chosen Era.

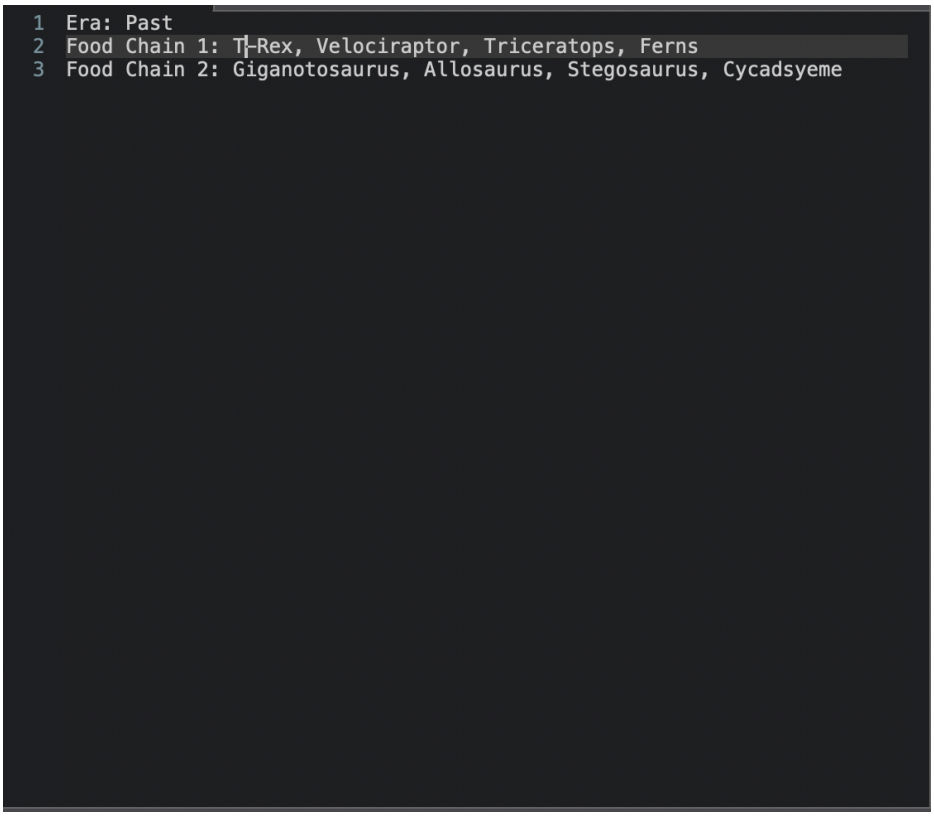

Figure 1 : The structure of an input file defining the hierarchy (past_animals.txt)

3)GUI Configuration (StartScreen): The StartScreen class serves as the gateway to the application, giving users full control over the simulation settings before it launches.

- **Grid Size:** Users can adjust the map dimensions (e.g., 15x15). This input directly updates the boardSize in the simulation model, scaling the world accordingly.
- **Rounds:** This setting establishes the duration of the simulation by defining the total number of turns (totalRounds).
- **Era:** Choosing an Era does two things: it switches the visual theme and instructs the Reader to load the specific set of creatures relevant to that timeline.

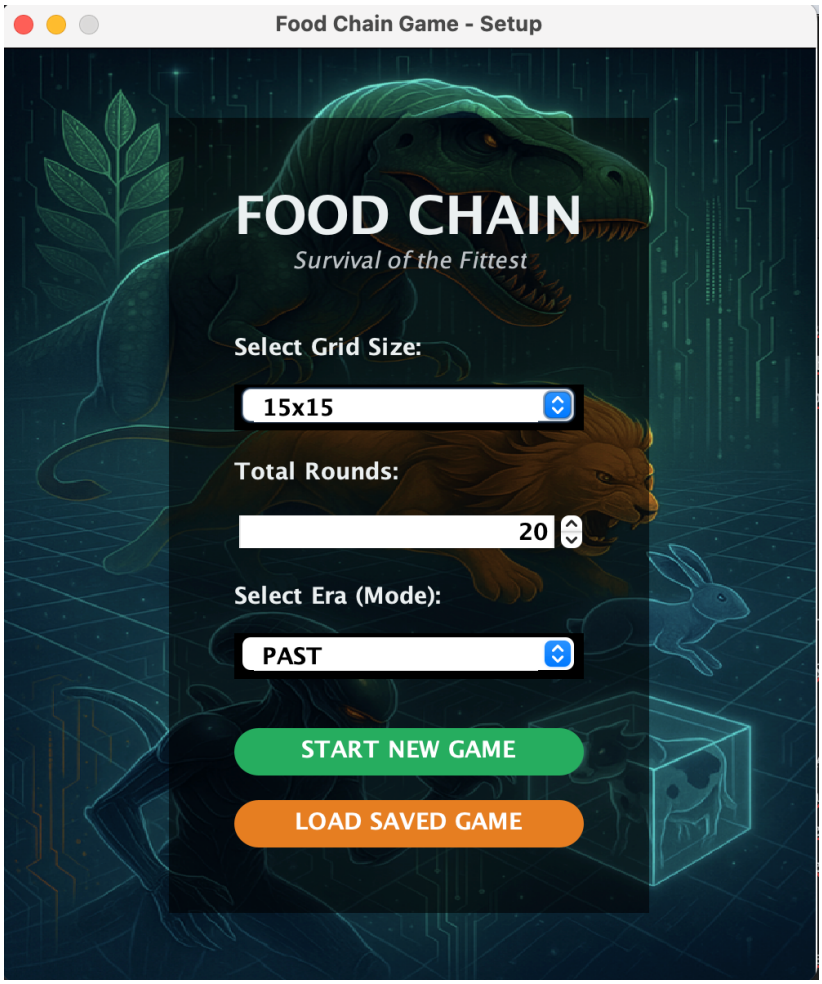

Figure 2: The configuration interface allowing users to select Era, Grid Size, and Rounds

3)Visual Representation: The Panel class handles the visual rendering of the game board using Graphics2D, creating an interface that adapts dynamically to the current game state.

- **Valid Moves (Green):** When the player selects the Predator, the system highlights adjacent, walkable tiles (1 unit distance) in green to indicate valid moves.
- **Special Abilities (Yellow):** If the special ability is ready (cooldown is 0), the extended movement range for the "Dash" ability is highlighted in yellow.

Movement Trails (Dash Lines): To make rapid movements easier to track, the system renders a **temporary colored line**connecting the start and end coordinates of an entity during a turn.

- **Cyan Line:** Represents the Player's (Predator) dash.
- **Red Line:** Represents the Apex Predator's movement.
- **Blue Line:** Represents the Prey's escape path. This visual cue helps the user follow the action, especially during long-distance jumps or teleports in the Future Era.

Figure 3: Normal moves (Green)

Figure 4: Special Abilities (Yellow)

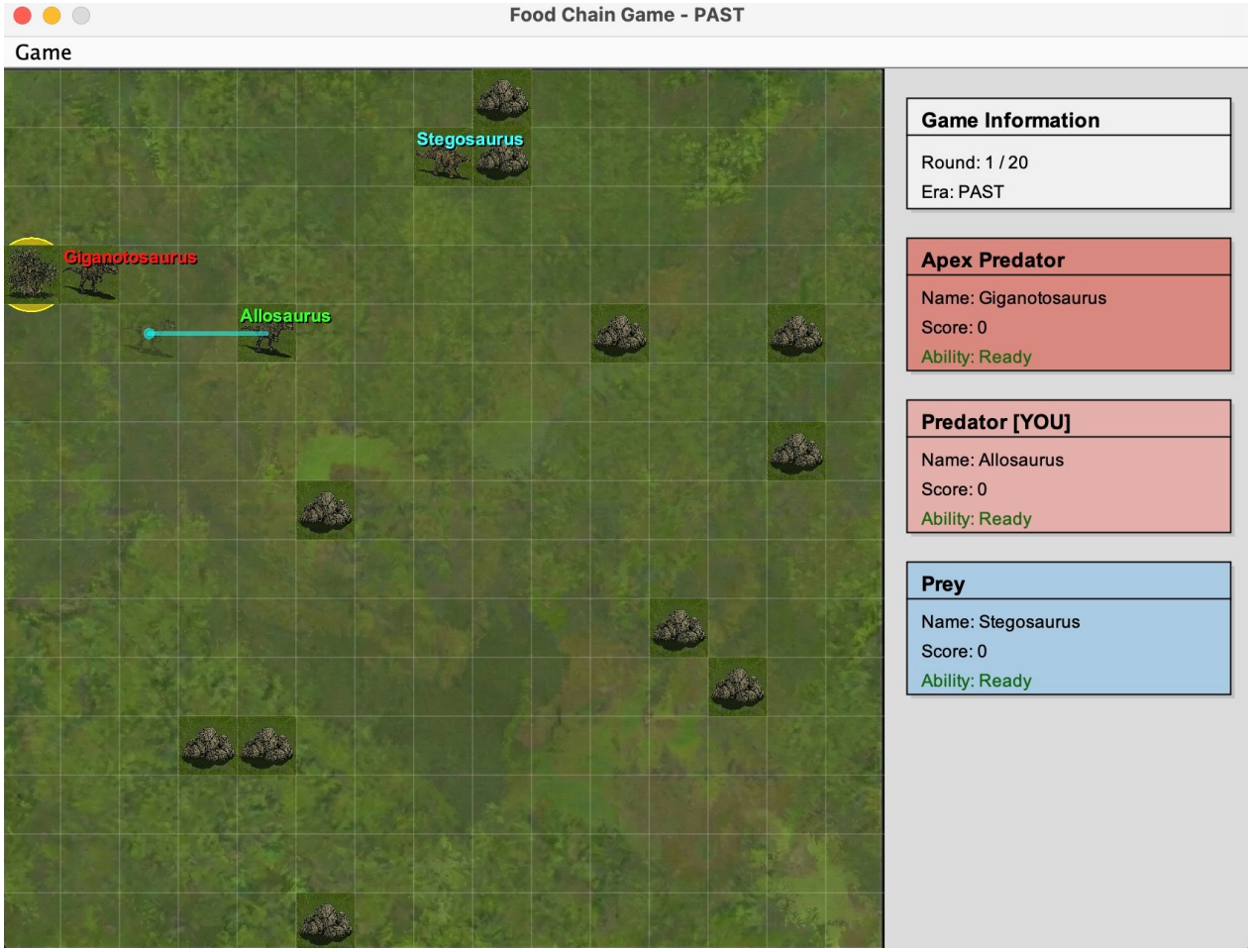

Figure 5:Movement Trails (Cyan Line)

4)Game Events (Eating and Respawning): When a Predator or Apex Predator moves to a coordinate occupied by a Prey, a collision event is triggered.

- **Visual Feedback:** A red "X" (deathLocations) marks the spot of the event.
- **Respawn:** The eaten entity immediately respawns at a random valid location, marked by a cyan circle (spawnLocations) with a "SPAWN" label.

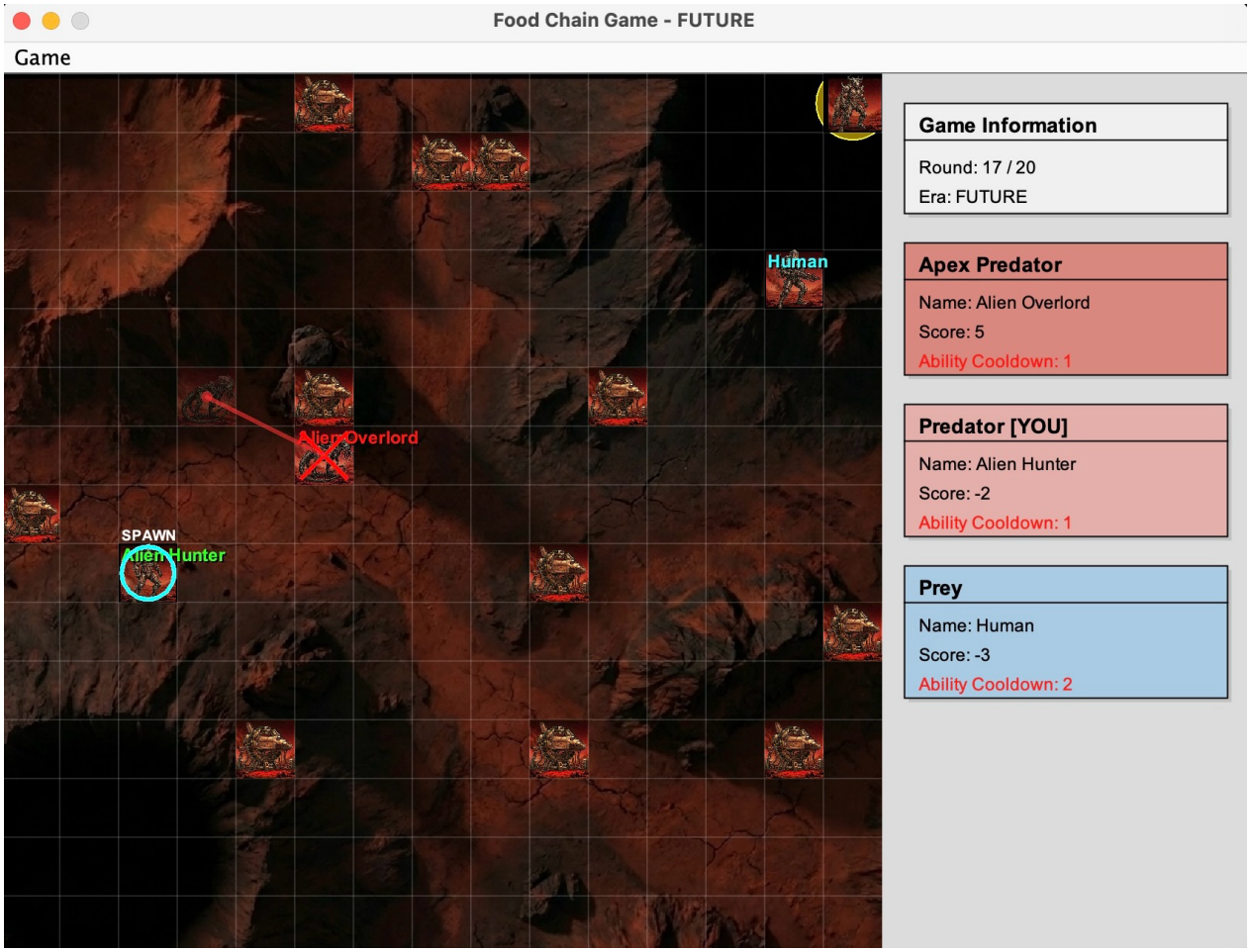

Figure 6: Death mark and Spawn label

5)Impact of Modifying Configuration Files (.txt): The application is designed with a **data-driven architecture**. This means the entity names and hierarchy are not hardcoded but loaded dynamically from text files. Modifying these files directly updates the game content without requiring code recompilation.

- **Scenario:** Changing the animal names in present_animals.txt.
- **Action:** We manually edited the file, changing "Lion" to "Tiger" and "Bunny" to "Rabbit".
- **Result:** Upon starting a new game in the Present Era, the Reader class parsed the modified file. The Dashboard and Log immediately reflected these new names ("Tiger", "Rabbit"), proving that the system dynamically instantiates objects based on string input.

```
1  Era: Present
2  Food Chain 1: Tiger, Cheetah, Rabbit, Grass
3  Food Chain 2: Polar Bear, Wolf, Deer, Shrubs
```

Figure 7: Changed file (present_animal.txt

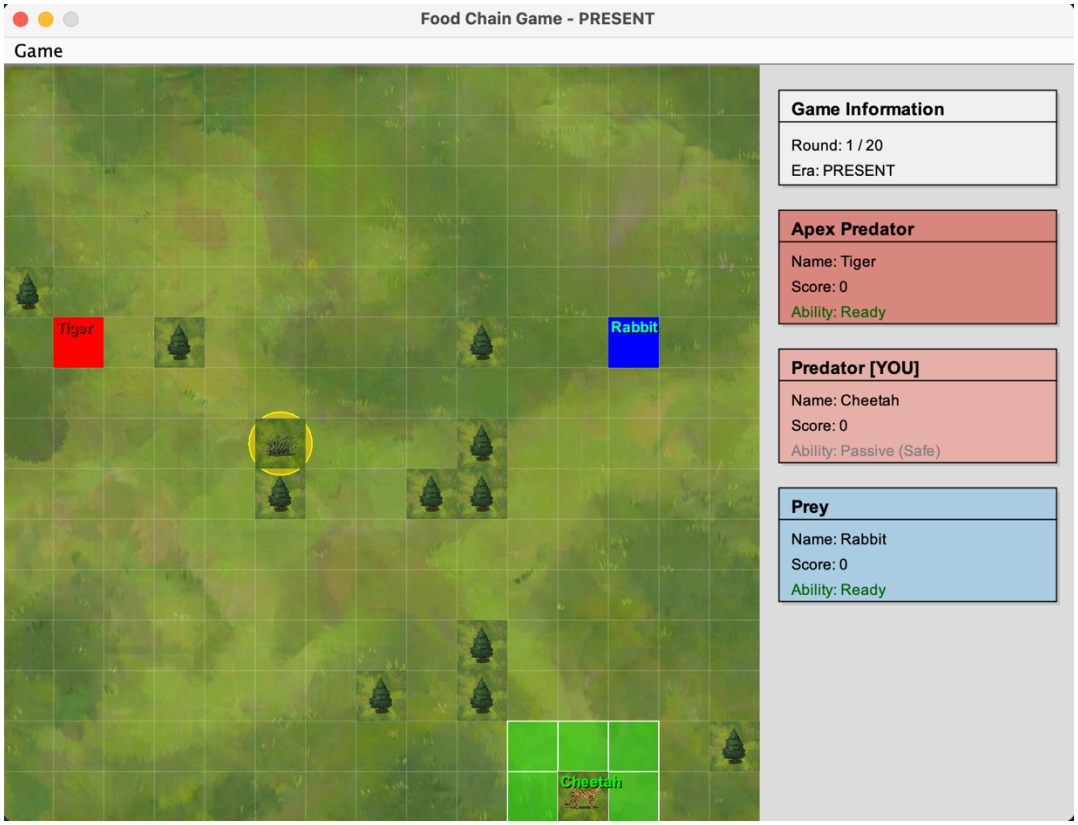

Figure 8: Changed Characters (present_animals.txt)

6)Logging System: The Log class records every significant state change with a timestamp to log.txt. This includes movement coordinates, score updates, and round transitions

```
[16:17:20] ==================================
[16:17:20] >>> Round 1 has begun.
[16:17:20] Map Loaded: src/foodchain/present_animals.txt | Selected: Food Chain 1: Tiger, Cheetah, Rabbit, Grass
[16:30:43] ==================================
[16:30:43]  NEW GAME SESSION STARTED
[16:30:43]  Era: PAST
[16:30:43]  Total Rounds: 10
[16:30:43]  Grid Size: 10x10
[16:30:43] ==================================
[16:30:43] >>> Round 1 has begun.
[16:30:43] Map Loaded: src/foodchain/past_animals.txt | Selected: Food Chain 2: Giganotosaurus, Allosaurus, Stegosaurus, Cycadsveme
[16:30:45] [MOVE] Player (Allosaurus) moved from (5,0) to (5,2)
[16:30:46] [MOVE] Stegosaurus moved from (6,8) to (6,7)
[16:30:46] [MOVE] Giganotosaurus moved from (1,6) to (3,6)
[16:30:46] <<< Round 1 terminated.
[16:30:46]
[16:30:46] >>> Round 2 has begun.
[16:30:47] [MOVE] Player (Allosaurus) moved from (5,2) to (6,3)
[16:30:47] [MOVE] Stegosaurus moved from (6,7) to (7,9)
[16:30:47] [MOVE] Giganotosaurus moved from (3,6) to (4,5)
[16:30:47] <<< Round 2 terminated.
[16:30:47]
[16:30:47] >>> Round 3 has begun.
[16:30:48] [MOVE] Player (Allosaurus) moved from (6,3) to (7,4)
[16:30:48] [MOVE] Stegosaurus moved from (7,9) to (6,8)
[16:30:48] [MOVE] Giganotosaurus moved from (4,5) to (5,4)
[16:30:48] <<< Round 3 terminated.
[16:30:48]
[16:30:48] >>> Round 4 has begun.
[16:30:49] [MOVE] Player (Allosaurus) moved from (7,4) to (7,6)
[16:30:49] [MOVE] Stegosaurus moved from (6,8) to (4,9)
[16:30:50] [MOVE] Giganotosaurus moved from (5,4) to (6,5)
[16:30:50] <<< Round 4 terminated.
[16:30:50]
[16:30:50] >>> Round 5 has begun.
[16:30:50] [MOVE] Player (Allosaurus) moved from (7,6) to (6,7)
[16:30:50] [MOVE] Stegosaurus moved from (4,9) to (3,9)
[16:30:50] [MOVE] Giganotosaurus moved from (6,5) to (6,7)
[16:30:50] [SCORE] Giganotosaurus GAINED 1 points. (Reason: Ate Allosaurus)
[16:30:50] [SCORE] Allosaurus LOST 1 points. (Reason: Eaten by Giganotosaurus)
[16:30:51] <<< Round 5 terminated.
[16:30:51]
[16:30:51] >>> Round 6 has begun.
[16:30:52] [MOVE] Player (Allosaurus) moved from (7,8) to (6,9)
[16:30:52] [MOVE] Stegosaurus moved from (3,9) to (1,8)
[16:30:53] [MOVE] Giganotosaurus moved from (6,7) to (6,8)
[16:30:53] <<< Round 6 terminated.
[16:30:53]
[16:30:53] >>> Round 7 has begun.
[16:30:53] [MOVE] Player (Allosaurus) moved from (6,9) to (4,9)
[16:30:54] [MOVE] Stegosaurus moved from (1,8) to (0,7)
[16:30:54] [MOVE] Giganotosaurus moved from (6,8) to (5,9)
[16:30:54] <<< Round 7 terminated.
[16:30:54]
[16:30:54] >>> Round 8 has begun.
[16:30:54] [MOVE] Player (Allosaurus) moved from (4,9) to (3,8)
[16:30:54] [MOVE] Stegosaurus moved from (0,7) to (0,6)
[16:30:54] [MOVE] Giganotosaurus moved from (5,9) to (4,8)
[16:30:55] <<< Round 8 terminated.
[16:30:55]
[16:30:55] >>> Round 9 has begun.
[16:30:55] [MOVE] Player (Allosaurus) moved from (3,8) to (2,8)
[16:30:56] [MOVE] Stegosaurus moved from (0,6) to (0,5)
[16:30:56] [MOVE] Giganotosaurus moved from (4,8) to (2,8)
[16:30:56] [SCORE] Giganotosaurus GAINED 1 points. (Reason: Ate Allosaurus)
[16:30:56] [SCORE] Allosaurus LOST 1 points. (Reason: Eaten by Giganotosaurus)
[16:30:57] <<< Round 9 terminated.
[16:30:57]
[16:30:57] >>> Round 10 has begun.
[16:30:57] [MOVE] Player (Allosaurus) moved from (7,2) to (6,3)
[16:30:57] [MOVE] Stegosaurus moved from (0,5) to (2,6)
[16:30:58] [MOVE] Giganotosaurus moved from (2,8) to (3,7)
[16:30:58] <<< Round 10 terminated.
```

.

```
[16:30:58] ****************************************
[16:30:58]  WINNER(S): Giganotosaurus (2 Pts)
[16:30:58] ****************************************
[16:30:58] === GAME OVER ===
[16:31:05] ==================================
[16:31:05]  NEW GAME SESSION STARTED
[16:31:05]  Era: PRESENT
[16:31:05]  Total Rounds: 20
[16:31:05]  Grid Size: 15x15
[16:31:05] ==================================
[16:31:05] >>> Round 1 has begun.
[16:31:05] Map Loaded: src/foodchain/present_animals.txt | Selected: Food Chain 1: Lion, Cheetah, Bunny, Grass
[16:31:07] [MOVE] Player (Cheetah) moved from (9,7) to (8,8)
[16:31:08] [MOVE] Bunny moved from (10,3) to (8,1)
[16:31:08] [MOVE] Lion moved from (10,5) to (10,8)
[16:31:09] <<< Round 1 terminated.
[16:31:09]
[16:31:09] >>> Round 2 has begun.
[16:31:09] [MOVE] Player (Cheetah) moved from (8,8) to (7,7)
[16:31:10] [MOVE] Bunny moved from (8,1) to (7,2)
[16:31:10] [MOVE] Lion moved from (10,8) to (9,7)
[16:31:10] <<< Round 2 terminated.
[16:31:10]
[16:31:10] >>> Round 3 has begun.
[16:31:10] [MOVE] Player (Cheetah) moved from (7,7) to (6,6)
[16:31:10] [MOVE] Bunny moved from (7,2) to (5,2)
[16:31:11] [MOVE] Lion moved from (9,7) to (6,7)
[16:31:11] <<< Round 3 terminated.
[16:31:11]
[16:31:11] >>> Round 4 has begun.
[16:31:12] [MOVE] Player (Cheetah) moved from (6,6) to (4,4)
[16:31:12] [MOVE] Bunny moved from (5,2) to (6,1)
[16:31:12] [MOVE] Lion moved from (6,7) to (5,6)
[16:31:12] <<< Round 4 terminated.
[16:31:12]
[16:31:12] >>> Round 5 has begun.
[16:31:13] [MOVE] Player (Cheetah) moved from (4,4) to (4,3)
[16:31:13] [MOVE] Bunny moved from (6,1) to (7,0)
[16:31:13] [MOVE] Lion moved from (5,6) to (4,5)
[16:31:14] <<< Round 5 terminated.
[16:31:14]
[16:31:14] >>> Round 6 has begun.
[16:31:14] [MOVE] Player (Cheetah) moved from (4,3) to (5,2)
[16:31:14] [MOVE] Bunny moved from (7,0) to (9,0)
[16:31:15] [MOVE] Lion moved from (4,5) to (4,2)
[16:31:15] <<< Round 6 terminated.
[16:31:15]
[16:31:15] >>> Round 7 has begun.
[16:31:16] [MOVE] Player (Cheetah) moved from (5,2) to (7,2)
[16:31:16] [MOVE] Bunny moved from (9,0) to (10,0)
[16:31:16] [MOVE] Lion moved from (4,2) to (5,2)
[16:31:16] <<< Round 7 terminated.
[16:31:16]
[16:31:16] >>> Round 8 has begun.
[16:31:17] [MOVE] Player (Cheetah) moved from (7,2) to (8,2)
[16:31:17] [MOVE] Bunny moved from (10,0) to (12,0)
[16:31:18] [MOVE] Lion moved from (5,2) to (8,2)
[16:31:18] [SCORE] Lion GAINED 1 points. (Reason: Ate Cheetah)
[16:31:18] [SCORE] Cheetah LOST 1 points. (Reason: Eaten by Lion)
[16:31:19] <<< Round 8 terminated.
[16:31:19]
[16:31:19] >>> Round 9 has begun.
[16:31:19] [MOVE] Player (Cheetah) moved from (4,3) to (3,3)
[16:31:19] [MOVE] Bunny moved from (12,0) to (11,1)
[16:31:19] [MOVE] Lion moved from (8,2) to (9,1)
[16:31:20] <<< Round 9 terminated.
[16:31:20]
[16:31:20] >>> Round 10 has begun.
[16:31:20] [MOVE] Player (Cheetah) moved from (3,3) to (4,4)
[16:31:21] [MOVE] Bunny moved from (11,1) to (13,3)
[16:31:21] [MOVE] Lion moved from (9,1) to (12,1)
[16:31:22] <<< Round 10 terminated.
[16:31:22]
```

[16:31:19]
[16:31:19] >>> Round 9 has begun.
[16:31:19] [MOVE] Player (Cheetah) moved from (4,3) to (3,3)
[16:31:19] [MOVE] Bunny moved from (12,0) to (11,1)
[16:31:19] [MOVE] Lion moved from (8,2) to (9,1)
[16:31:20] <<< Round 9 terminated.
[16:31:20]
[16:31:20] >>> Round 10 has begun.
[16:31:20] [MOVE] Player (Cheetah) moved from (3,3) to (4,4)
[16:31:21] [MOVE] Bunny moved from (11,1) to (13,3)
[16:31:21] [MOVE] Lion moved from (9,1) to (12,1)
[16:31:22] <<< Round 10 terminated.
[16:31:22]
[16:31:22] >>> Round 11 has begun.
[16:31:22] [MOVE] Player (Cheetah) moved from (4,4) to (5,4)
[16:31:22] [MOVE] Bunny moved from (13,3) to (14,4)
[16:31:22] [MOVE] Lion moved from (12,1) to (13,2)
[16:31:23] <<< Round 11 terminated.
[16:31:23]
[16:31:23] >>> Round 12 has begun.
[16:31:23] [MOVE] Player (Cheetah) moved from (5,4) to (6,4)
[16:31:23] [MOVE] Bunny moved from (14,4) to (12,6)
[16:31:24] [MOVE] Lion moved from (13,2) to (13,5)
[16:31:24] <<< Round 12 terminated.
[16:31:24]
[16:31:24] >>> Round 13 has begun.
[16:31:25] [MOVE] Player (Cheetah) moved from (6,4) to (7,4)
[16:31:25] [MOVE] Bunny moved from (12,6) to (11,7)
[16:31:25] [MOVE] Lion moved from (13,5) to (12,6)
[16:31:25] <<< Round 13 terminated.
[16:31:25]
[16:31:25] >>> Round 14 has begun.
[16:31:25] [MOVE] Player (Cheetah) moved from (7,4) to (8,5)
[16:31:26] [MOVE] Bunny moved from (11,7) to (9,9)
[16:31:26] [MOVE] Lion moved from (12,6) to (9,6)
[16:31:27] <<< Round 14 terminated.
[16:31:27]
[16:31:27] >>> Round 15 has begun.
[16:31:27] [MOVE] Player (Cheetah) moved from (8,5) to (6,7)
[16:31:28] [MOVE] Bunny moved from (9,9) to (8,10)
[16:31:28] [MOVE] Lion moved from (9,6) to (8,7)
[16:31:28] <<< Round 15 terminated.
[16:31:28]
[16:31:28] >>> Round 16 has begun.
[16:31:28] [MOVE] Player (Cheetah) moved from (6,7) to (5,8)
[16:31:28] [MOVE] Bunny moved from (8,10) to (6,12)
[16:31:29] [MOVE] Lion moved from (8,7) to (5,7)
[16:31:29] <<< Round 16 terminated.
[16:31:29]
[16:31:29] >>> Round 17 has begun.
[16:31:30] [MOVE] Player (Cheetah) moved from (5,8) to (5,10)
[16:31:31] [MOVE] Bunny moved from (6,12) to (7,13)
[16:31:31] [MOVE] Lion moved from (5,7) to (5,8)
[16:31:31] <<< Round 17 terminated.
[16:31:31]
[16:31:31] >>> Round 18 has begun.
[16:31:31] [MOVE] Player (Cheetah) moved from (5,10) to (6,11)
[16:31:32] [MOVE] Bunny moved from (7,13) to (8,14)
[16:31:32] [MOVE] Lion moved from (5,8) to (6,9)
[16:31:32] <<< Round 18 terminated.
[16:31:32]
[16:31:32] >>> Round 19 has begun.
[16:31:32] [MOVE] Player (Cheetah) moved from (6,11) to (7,12)
[16:31:33] [MOVE] Bunny moved from (8,14) to (10,14)
[16:31:33] [MOVE] Lion moved from (6,9) to (6,12)
[16:31:34] <<< Round 19 terminated.
[16:31:34]
[16:31:34] >>> Round 20 has begun.
[16:31:34] [MOVE] Player (Cheetah) moved from (7,12) to (8,12)
[16:31:34] [MOVE] Bunny moved from (10,14) to (11,14)
[16:31:34] [MOVE] Lion moved from (6,12) to (7,12)
[16:31:34] <<< Round 20 terminated.
[16:31:34]

```
[16:31:34] ************************************
[16:31:34]  WINNER(S): Lion (1 Pts)
[16:31:34] ************************************
[16:31:34] == GAME OVER ==
[16:31:57] ==================================
[16:31:57]  NEW GAME SESSION STARTED
[16:31:57]  Era: FUTURE
[16:31:57]  Total Rounds: 30
[16:31:57]  Grid Size: 20x20
[16:31:57] ==================================
[16:31:57] >>> Round 1 has begun.
[16:31:57] Map Loaded: src/foodchain/future_animals.txt | Selected: Food Chain 1: Alien Overlord, Alien Hunter, Human, Cow
[16:32:00] [MOVE] Player (Alien Hunter) moved from (15,9) to (13,11)
[16:32:01] [MOVE] Human moved from (4,14) to (4,11)
[16:32:01] [MOVE] Alien Overlord moved from (19,18) to (16,15)
[16:32:02] <<< Round 1 terminated.
[16:32:02]
[16:32:02] >>> Round 2 has begun.
[16:32:02] [MOVE] Player (Alien Hunter) moved from (13,11) to (12,11)
[16:32:03] [MOVE] Human moved from (4,11) to (5,10)
[16:32:03] [MOVE] Alien Overlord moved from (16,15) to (16,14)
[16:32:03] <<< Round 2 terminated.
[16:32:03]
[16:32:03] >>> Round 3 has begun.
[16:32:03] [MOVE] Player (Alien Hunter) moved from (12,11) to (11,10)
[16:32:03] [MOVE] Human moved from (5,10) to (8,10)
[16:32:04] [MOVE] Alien Overlord moved from (16,14) to (13,11)
[16:32:04] <<< Round 3 terminated.
[16:32:04]
[16:32:04] >>> Round 4 has begun.
[16:32:05] [MOVE] Player (Alien Hunter) moved from (11,10) to (9,10)
[16:32:05] [MOVE] Human moved from (8,10) to (7,9)
[16:32:05] [MOVE] Alien Overlord moved from (13,11) to (13,10)
[16:32:05] <<< Round 4 terminated.
[16:32:05]
[16:32:05] >>> Round 5 has begun.
[16:32:06] [MOVE] Player (Alien Hunter) moved from (9,10) to (8,9)
[16:32:06] [MOVE] Human moved from (7,9) to (4,12)
[16:32:07] [MOVE] Alien Overlord moved from (13,10) to (10,9)
[16:32:07] <<< Round 5 terminated.
[16:32:07]
[16:32:07] >>> Round 6 has begun.
[16:32:08] [MOVE] Player (Alien Hunter) moved from (8,9) to (7,10)
[16:32:08] [MOVE] Human moved from (4,12) to (5,11)
[16:32:08] [MOVE] Alien Overlord moved from (10,9) to (9,10)
[16:32:08] <<< Round 6 terminated.
[16:32:08]
[16:32:08] >>> Round 7 has begun.
[16:32:10] [MOVE] Player (Alien Hunter) moved from (7,10) to (5,10)
[16:32:11] [MOVE] Human moved from (5,11) to (8,14)
[16:32:11] [MOVE] Alien Overlord moved from (9,10) to (6,10)
[16:32:12] <<< Round 7 terminated.
[16:32:12]
[16:32:12] >>> Round 8 has begun.
[16:32:13] [MOVE] Player (Alien Hunter) moved from (5,10) to (5,11)
[16:32:13] [MOVE] Human moved from (8,14) to (9,13)
[16:32:13] [MOVE] Alien Overlord moved from (6,10) to (5,11)
[16:32:13] [SCORE] Alien Overlord GAINED 1 points. (Reason: Ate Alien Hunter)
[16:32:13] [SCORE] Alien Hunter LOST 1 points. (Reason: Eaten by Alien Overlord)
[16:32:13] <<< Round 8 terminated.
[16:32:13]
[16:32:13] >>> Round 9 has begun.
[16:32:14] [MOVE] Player (Alien Hunter) moved from (2,8) to (3,7)
[16:32:14] [MOVE] Human moved from (9,13) to (12,13)
[16:32:15] [MOVE] Alien Overlord moved from (5,11) to (2,8)
[16:32:15] <<< Round 9 terminated.
[16:32:15]
[16:32:15] >>> Round 10 has begun.
[16:32:16] [MOVE] Player (Alien Hunter) moved from (3,7) to (5,7)
[16:32:16] [MOVE] Human moved from (12,13) to (13,12)
[16:32:16] [MOVE] Alien Overlord moved from (2,8) to (3,7)
[16:32:16] <<< Round 10 terminated.
[16:32:16]
```

[16:32:16] >>> Round 11 has begun.
[16:32:17] [MOVE] Player (Alien Hunter) moved from (5,7) to (6,8)
[16:32:18] [MOVE] Human moved from (13,12) to (16,9)
[16:32:18] [MOVE] Alien Overlord moved from (3,7) to (6,8)
[16:32:18] [SCORE] Alien Overlord GAINED 1 points. (Reason: Ate Alien Hunter)
[16:32:18] [SCORE] Alien Hunter LOST 1 points. (Reason: Eaten by Alien Overlord)
[16:32:19] <<< Round 11 terminated.
[16:32:19]
[16:32:19] >>> Round 12 has begun.
[16:32:19] [MOVE] Player (Alien Hunter) moved from (13,0) to (14,1)
[16:32:20] [MOVE] Human moved from (16,9) to (17,9)
[16:32:20] [MOVE] Alien Overlord moved from (6,8) to (7,9)
[16:32:20] <<< Round 12 terminated.
[16:32:20]
[16:32:20] >>> Round 13 has begun.
[16:32:20] [MOVE] Player (Alien Hunter) moved from (14,1) to (16,3)
[16:32:21] [MOVE] Human moved from (17,9) to (18,9)
[16:32:21] [MOVE] Alien Overlord moved from (7,9) to (10,9)
[16:32:21] <<< Round 13 terminated.
[16:32:21]
[16:32:21] >>> Round 14 has begun.
[16:32:22] [MOVE] Player (Alien Hunter) moved from (16,3) to (17,4)
[16:32:22] [MOVE] Human moved from (18,9) to (19,9)
[16:32:22] [SCORE] Human GAINED 3 points. (Reason: Ate Food)
[16:32:22] [MOVE] Alien Overlord moved from (10,9) to (11,9)
[16:32:23] <<< Round 14 terminated.
[16:32:23]
[16:32:23] >>> Round 15 has begun.
[16:32:23] [MOVE] Player (Alien Hunter) moved from (17,4) to (18,5)
[16:32:23] [MOVE] Human moved from (19,9) to (16,6)
[16:32:24] [MOVE] Alien Overlord moved from (11,9) to (14,6)
[16:32:24] <<< Round 15 terminated.
[16:32:24]
[16:32:24] >>> Round 16 has begun.
[16:32:25] [MOVE] Player (Alien Hunter) moved from (18,5) to (16,3)
[16:32:25] [MOVE] Human moved from (16,6) to (17,7)
[16:32:25] [MOVE] Alien Overlord moved from (14,6) to (14,7)
[16:32:26] <<< Round 16 terminated.
[16:32:26]
[16:32:26] >>> Round 17 has begun.
[16:32:26] [MOVE] Player (Alien Hunter) moved from (16,3) to (17,4)
[16:32:27] [MOVE] Human moved from (17,7) to (17,10)
[16:32:27] [MOVE] Alien Overlord moved from (14,7) to (17,4)
[16:32:27] [SCORE] Alien Overlord GAINED 1 points. (Reason: Ate Alien Hunter)
[16:32:27] [SCORE] Alien Hunter LOST 1 points. (Reason: Eaten by Alien Overlord)
[16:32:28] <<< Round 17 terminated.
[16:32:28]
[16:32:28] >>> Round 18 has begun.
[16:32:28] [MOVE] Player (Alien Hunter) moved from (10,16) to (11,15)
[16:32:29] [MOVE] Human moved from (17,10) to (16,9)
[16:32:29] [MOVE] Alien Overlord moved from (17,4) to (16,5)
[16:32:29] <<< Round 18 terminated.
[16:32:29]
[16:32:29] >>> Round 19 has begun.
[16:32:30] [MOVE] Player (Alien Hunter) moved from (11,15) to (13,13)
[16:32:30] [MOVE] Human moved from (16,9) to (13,12)
[16:32:30] [MOVE] Alien Overlord moved from (16,5) to (13,8)
[16:32:31] <<< Round 19 terminated.
[16:32:31]
[16:32:31] >>> Round 20 has begun.
[16:32:31] [MOVE] Player (Alien Hunter) moved from (13,13) to (12,12)
[16:32:32] [MOVE] Human moved from (13,12) to (14,11)
[16:32:32] [MOVE] Alien Overlord moved from (13,8) to (14,9)
[16:32:32] <<< Round 20 terminated.
[16:32:32]
[16:32:32] >>> Round 21 has begun.
[16:32:33] [MOVE] Player (Alien Hunter) moved from (12,12) to (13,11)
[16:32:33] [MOVE] Human moved from (14,11) to (17,8)
[16:32:33] [MOVE] Alien Overlord moved from (14,9) to (13,11)
[16:32:33] [SCORE] Alien Overlord GAINED 1 points. (Reason: Ate Alien Hunter)
[16:32:33] [SCORE] Alien Hunter LOST 1 points. (Reason: Eaten by Alien Overlord)
[16:32:34] <<< Round 21 terminated.

```
[16:32:32]
[16:32:32] >>> Round 21 has begun.
[16:32:33] [MOVE] Player (Alien Hunter) moved from (12,12) to (13,11)
[16:32:33] [MOVE] Human moved from (14,11) to (17,8)
[16:32:33] [MOVE] Alien Overlord moved from (14,9) to (13,11)
[16:32:33] [SCORE] Alien Overlord GAINED 1 points. (Reason: Ate Alien Hunter)
[16:32:33] [SCORE] Alien Hunter LOST 1 points. (Reason: Eaten by Alien Overlord)
[16:32:34] <<< Round 21 terminated.
[16:32:34]
[16:32:34] >>> Round 22 has begun.
[16:32:35] [MOVE] Player (Alien Hunter) moved from (10,10) to (12,8)
[16:32:35] [MOVE] Human moved from (17,8) to (16,7)
[16:32:35] [MOVE] Alien Overlord moved from (13,11) to (13,10)
[16:32:35] <<< Round 22 terminated.
[16:32:35]
[16:32:35] >>> Round 23 has begun.
[16:32:36] [MOVE] Player (Alien Hunter) moved from (12,8) to (13,7)
[16:32:36] [MOVE] Human moved from (16,7) to (19,4)
[16:32:36] [MOVE] Alien Overlord moved from (13,10) to (13,7)
[16:32:36] [SCORE] Alien Overlord GAINED 1 points. (Reason: Ate Alien Hunter)
[16:32:36] [SCORE] Alien Hunter LOST 1 points. (Reason: Eaten by Alien Overlord)
[16:32:37] <<< Round 23 terminated.
[16:32:37]
[16:32:37] >>> Round 24 has begun.
[16:32:38] [MOVE] Player (Alien Hunter) moved from (14,18) to (15,17)
[16:32:38] [MOVE] Human moved from (19,4) to (18,3)
[16:32:38] [MOVE] Alien Overlord moved from (13,7) to (14,6)
[16:32:38] <<< Round 24 terminated.
[16:32:38]
[16:32:38] >>> Round 25 has begun.
[16:32:39] [MOVE] Player (Alien Hunter) moved from (15,17) to (13,15)
[16:32:40] [MOVE] Human moved from (18,3) to (15,3)
[16:32:40] [MOVE] Alien Overlord moved from (14,6) to (15,3)
[16:32:40] [SCORE] Alien Overlord GAINED 1 points. (Reason: Ate Human)
[16:32:40] [SCORE] Human LOST 1 points. (Reason: Eaten by Alien Overlord)
[16:32:41] <<< Round 25 terminated.
[16:32:41]
[16:32:41] >>> Round 26 has begun.
[16:32:41] [MOVE] Player (Alien Hunter) moved from (13,15) to (12,14)
[16:32:42] [MOVE] Human moved from (6,7) to (5,6)
[16:32:42] [MOVE] Alien Overlord moved from (15,3) to (14,4)
[16:32:42] <<< Round 26 terminated.
[16:32:42]
[16:32:42] >>> Round 27 has begun.
[16:32:43] [MOVE] Player (Alien Hunter) moved from (12,14) to (11,13)
[16:32:43] [MOVE] Human moved from (5,6) to (2,3)
[16:32:44] [MOVE] Alien Overlord moved from (14,4) to (11,7)
[16:32:44] <<< Round 27 terminated.
[16:32:44]
[16:32:44] >>> Round 28 has begun.
[16:32:45] [MOVE] Player (Alien Hunter) moved from (11,13) to (9,11)
[16:32:45] [MOVE] Human moved from (2,3) to (1,2)
[16:32:45] [MOVE] Alien Overlord moved from (11,7) to (10,8)
[16:32:45] <<< Round 28 terminated.
[16:32:45]
[16:32:45] >>> Round 29 has begun.
[16:32:46] [MOVE] Player (Alien Hunter) moved from (9,11) to (8,12)
[16:32:46] [MOVE] Human moved from (1,2) to (0,2)
[16:32:46] [SCORE] Human GAINED 3 points. (Reason: Ate Food)
[16:32:46] [MOVE] Alien Overlord moved from (10,8) to (8,11)
[16:32:47] <<< Round 29 terminated.
[16:32:47]
[16:32:47] >>> Round 30 has begun.
[16:32:47] [MOVE] Player (Alien Hunter) moved from (8,12) to (7,13)
[16:32:48] [MOVE] Human moved from (0,2) to (3,5)
[16:32:48] [MOVE] Alien Overlord moved from (8,11) to (7,12)
[16:32:48] <<< Round 30 terminated.
[16:32:48]
[16:32:48] ****************************************
[16:32:48]  WINNER(S): Alien Overlord (6 Pts)
[16:32:48] ****************************************
[16:32:48] === GAME OVER ===
```

7)Save/Load Functionality: The user can save the exact state of the board.

The Simulation class serializes the animals, obstacles, and food lists into a custom text format.

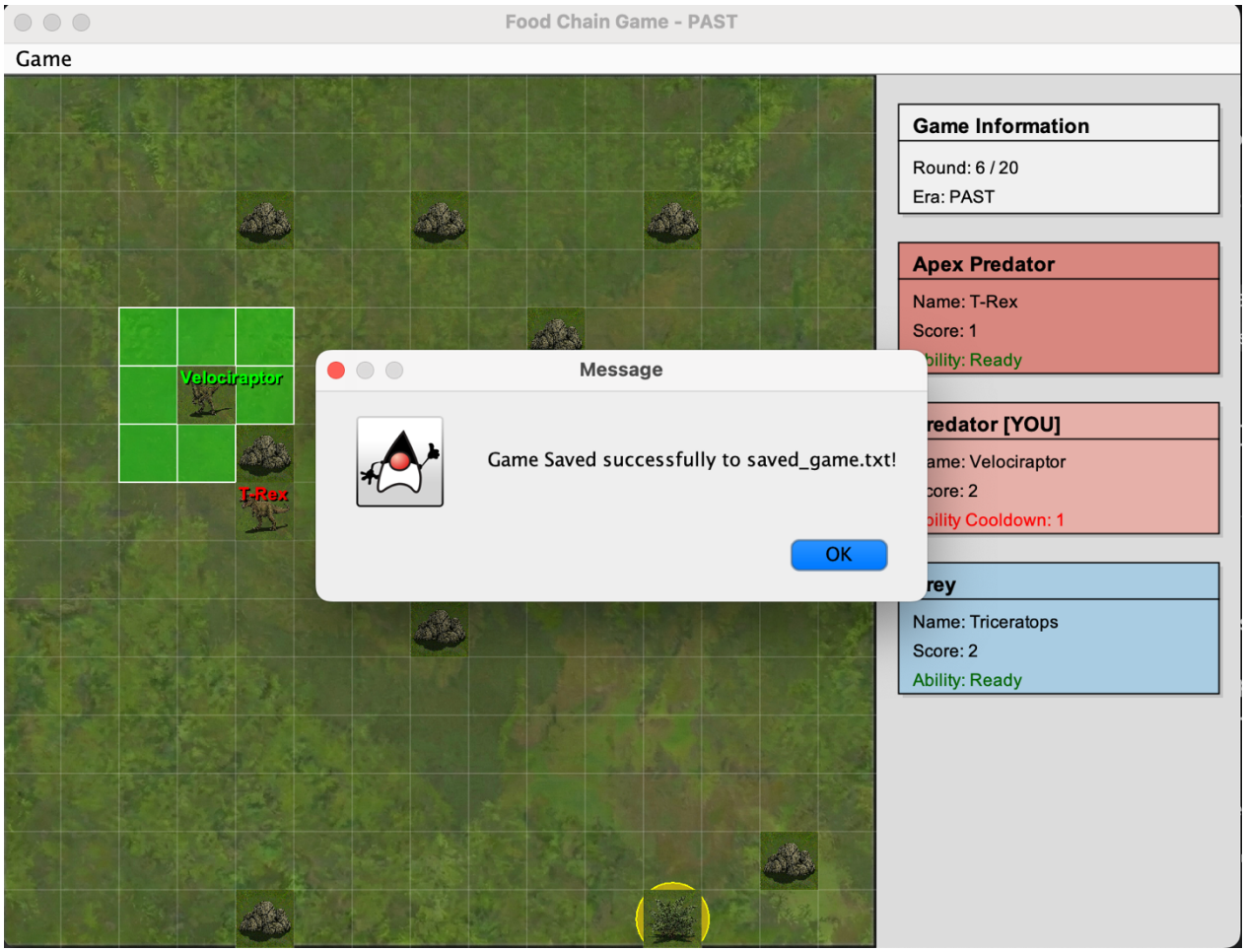

Figure 9: If you press the ‘S’ key on the keyboard, the game will save

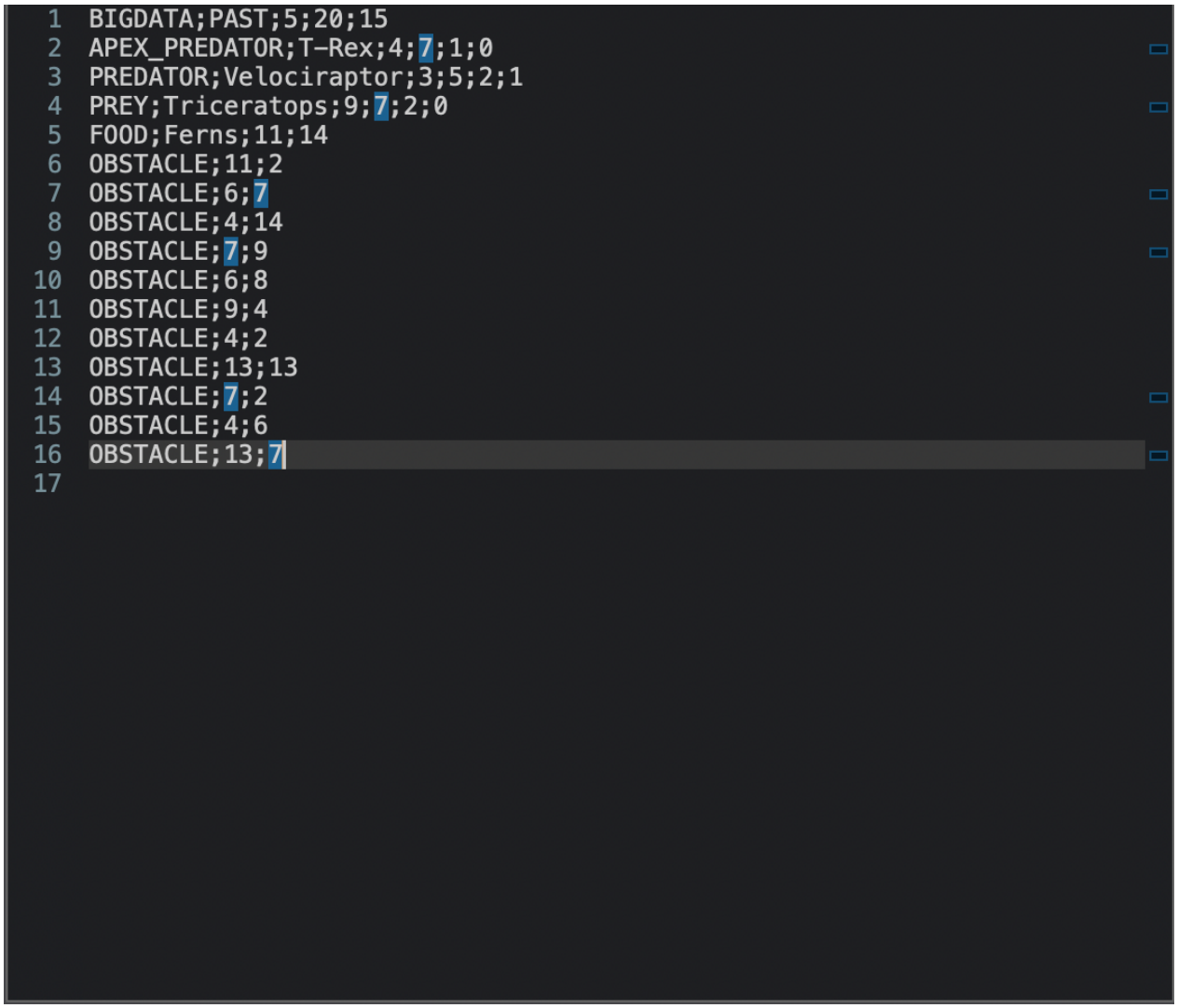

Figure 10: saved_game.txt file

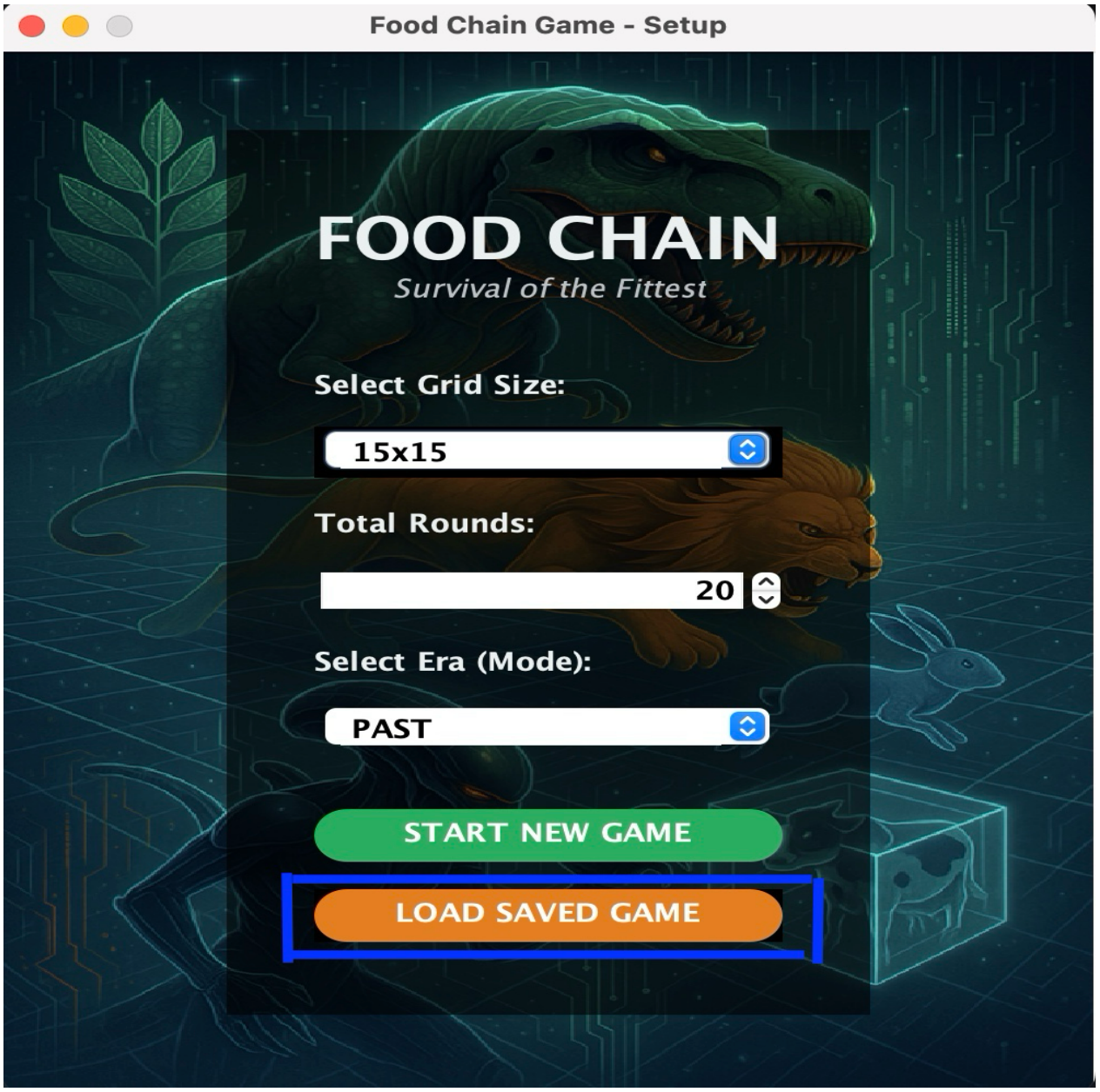

Figure 11: If you click the 'LOAD SAVED GAME' button, the saved game will be loaded

Figure 12: Saved game

8) Impact of Modifying Saved Games: The save system allows for adding new entities by simply inserting new lines into the text file. This demonstrates that the loadGameFromText method dynamically reconstructs the list of animals without a fixed limit.

```
BIGDATA;PAST;5;20;15
APEX_PREDATOR;T-Rex;4;7;1;0
PREDATOR;Velociraptor;3;5;2;1
PREY;Triceratops;9;7;2;0
PREY;Triceratops;9;8;2;0
PREY;Triceratops;9;10;2;0
PREY;Triceratops;11;7;2;0
FOOD;Ferns;11;14
OBSTACLE;11;2
OBSTACLE;6;7
OBSTACLE;4;14
OBSTACLE;7;9
OBSTACLE;6;8
OBSTACLE;9;4
OBSTACLE;4;2
OBSTACLE;13;13
OBSTACLE;7;2
OBSTACLE;4;6
OBSTACLE;13;7
```

Figure 13: We changed number of preys

```
BIGDATA;PAST;5;20;15
APEX_PREDATOR;T-Rex;4;7;1;0
PREDATOR;Velociraptor;3;5;2;1
PREY;Triceratops;9;7;2;0
FOOD;Ferns;11;14
FOOD;Ferns;13;14
FOOD;Ferns;11;12
FOOD;Ferns;13;12
OBSTACLE;11;2
OBSTACLE;6;7
OBSTACLE;4;14
OBSTACLE;7;9
OBSTACLE;6;8
OBSTACLE;9;4
OBSTACLE;4;2
OBSTACLE;13;13
OBSTACLE;7;2
OBSTACLE;4;6
OBSTACLE;13;7
```

Figure 14: We changed number of foods

9)Game Completion and Results: The simulation concludes when either the maximum number of rounds is reached or the Player (Predator) is eliminated by the Apex Predator. Unlike standard Java message boxes, this project features a **custom-designed Game Over screen**.

- **Dynamic Visuals:** The screen dynamically identifies the winner. If a specific animal wins (e.g., Lion), the background of the dialog changes to that animal's thematic image.
- **Scoreboard:** It displays the final winner and their score, along with the player's performance.
- **Controls:** The user is presented with styled buttons to **"Play Again"** (which resets the Simulation model and reloading assets) or **"Exit".**

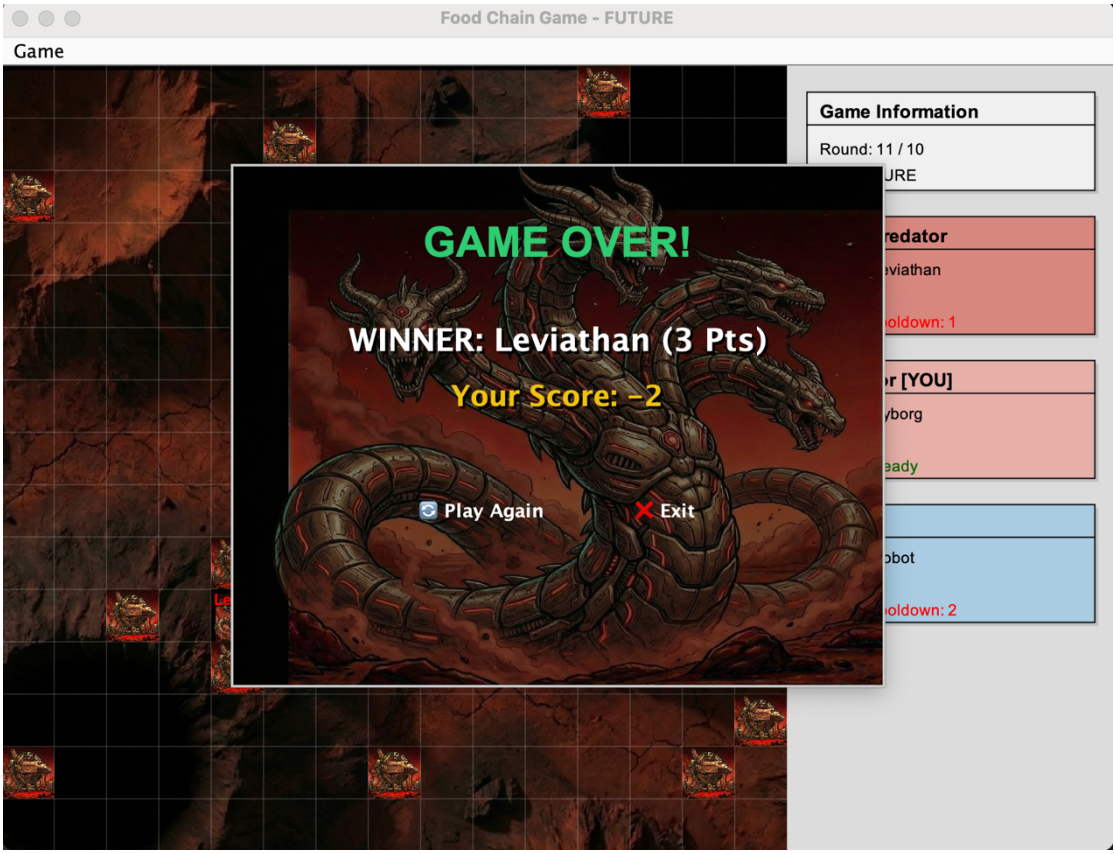

Figure 15: Game Over Screen (Win)

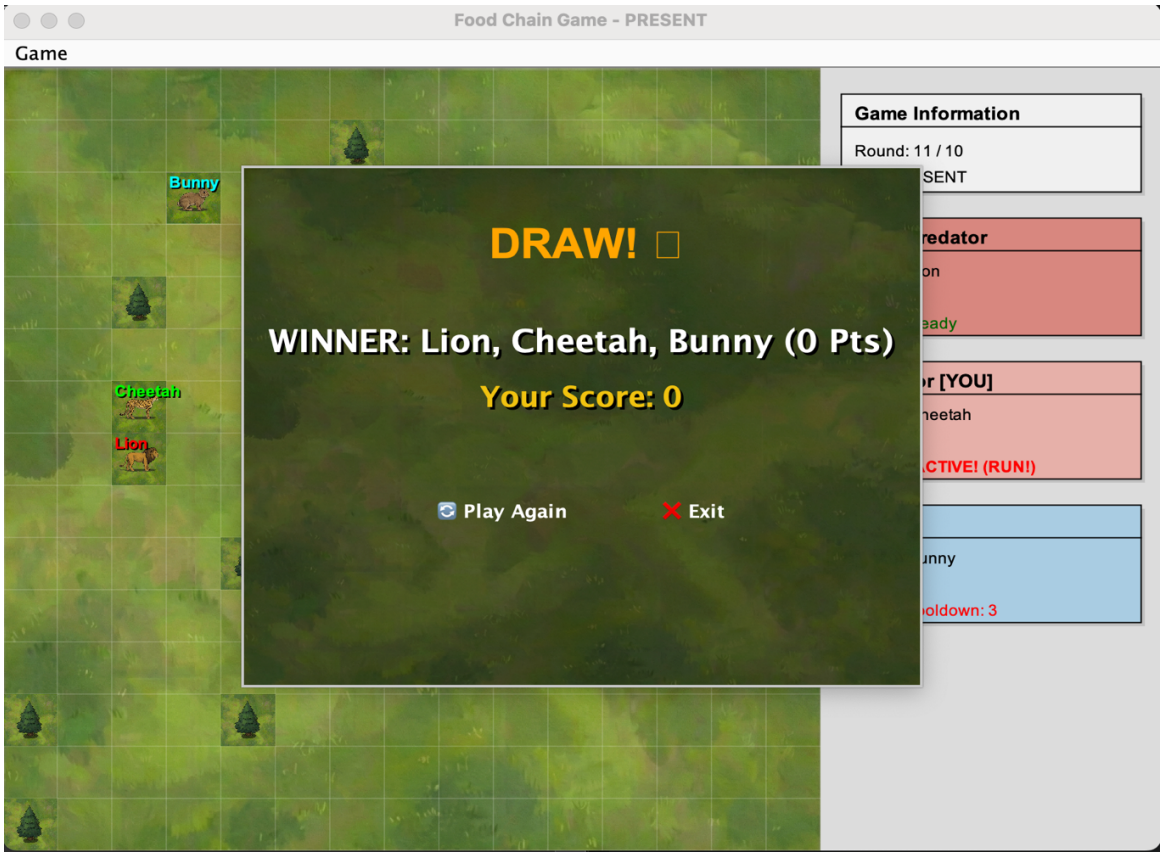

Figure 16: Game Over Screen (Draw)

Part 2: Project Design Description

1.Package: foodchain.model

1.1. Class: Simulation (The Model Engine) This class acts as the central controller for the game state, managing entity lists, time flow, and rule enforcement. It implements Serializable to support the save/load system.

- Simulation(int boardSize, int totalRounds, Animal.Era era): The constructor initializes the game environment. It instantiates the dynamic lists (animals, foods, obstacles, deathLocations) and triggers the initial setup via initializeGame().

- MovesRecorder (Inner Class): A static inner class implementing Serializable. It acts as a data structure to record the starting and ending coordinates of an animal's movement (x, y to z, w) along with its type. This is used by the View layer to render visual movement trails.
- initializeGame(): Resets the simulation to its starting state.

1. Clears all entity lists (animals.clear(), foods.clear()).
2. Calls Logger.logSetup to record session metadata.
3. Generates random obstacles using generateObstacles().
4. Populates the board with specific animals (Apex, Predator, Prey) based on the selected Era (Past, Present, or Future).
5. Adds the initial food supply and randomizes positions via initializePositions().

- finishPreyMoves(): Orchestrates the AI turn for all Prey entities.

- Iterates through the animals list to find instances of Prey.
- Reduces ability cooldowns using reduceCooldown().
- Identifies the nearest food source using the getClosestFood() helper method.
- Executes the move() method, passing the Player and the target Food as parameters.
- Logs the movement and records it for visual trail rendering.

- apexMoves(): Orchestrates the AI turn for Apex Predators.

- Prioritizes the Player (Predator) as the primary target. If the Player is null or distant, it switches focus to the nearest Prey using getClosestPrey().
- Executes the movement logic and logs the new coordinates.

- finalizeRound(): Manages the temporal progression of the game.

- Increments the currentRound counter.
- Logs the end of the current round and the start of the next one.
- Checks if currentRound >= totalRounds. If true, it triggers calculateWinner() to end the session.

- checkEatings(): The core Collision Detection Engine.

- Animal-vs-Animal: Uses a nested loop (O(N^2) complexity) to check coordinate equality (a1.x == a2.x && a1.y == a2.y) between all animals. It enforces the food chain hierarchy (Apex > Predator > Prey) and triggers eatLogic().
- Animal-vs-Food: Iterates through the foods list. If a Prey occupies the same cell as a Food object, it awards points (+3), logs the event, adds the location to deathLocations for visual feedback, and calls respawn(food).

- eatLogic(Animal hunter, Animal victim, int points): Handles the transactional logic of predation.

- Updates scores: Hunter gains points, Victim loses 1 point.
- Logs the event (e.g., "Lion ate Bunny").
- Records the victim's location for the "Red X" visual effect.
- Immediately respawns the victim at a new random location.

- respawn(Object type): Implements a Safe Spawning Algorithm.

- Uses java.security.SecureRandom to generate random (x, y) coordinates.
- Utilizes a do-while loop that checks isCellFull(x, y) to ensure the new coordinate is not occupied by any other Animal, Food, or Obstacle.

- isCellFull(int x, int y): A validation helper. Returns true if the specified coordinate contains any entity (Animal, Food list, or Obstacle list). Used to prevent overlapping spawns.
- generateObstacles(): Randomly populates the board with obstacles. The density is calculated as (boardSize * boardSize) / 20 to ensure a balanced difficulty.
- **hasObstacle(int x, int y):** Spatial Query. Iterates through the obstacles list to determine if a specific coordinate is blocked.
- saveGameToText(String filename): Serialization Logic. Writes the current game state to a structured text file using BufferedWriter.

- Writes the "BIGDATA" header containing the Era, Round, and Board Size.

- Iterates through animals, foods, and obstacles lists, writing their properties (Type, Name, X, Y, Score, CD) line-by-line.

- loadGameFromText(String filename): Deserialization Logic. Reads the text file using BufferedReader.

- Parses the header to initialize a new Simulation object.
- Reads the file line-by-line, splitting strings by the semicolon delimiter (;) to reconstruct Animal, Food, and Obstacle objects and add them to their respective lists.

- getClosestFood(Animal a): AI Sensing Helper. Iterates through the foods list and calculates the Manhattan Distance (|x1-x2| + |y1-y2|) to the animal. Returns the Food object with the minimum distance.
- getClosestPrey(Animal apex): AI Sensing Helper for Apex. Scans all animals to find the nearest Prey instance using distance minimization logic.
- calculateWinner(): Triggered at the end of the game. Calls getWinnerInfo() to format the result string and logs the game over event.
- isApexClose(Animal currentAnimal): A proximity check used specifically for the Predator's "Walk+Walk" logic in the Present Era. Returns true if an Apex Predator is within a 1-tile radius.
- initializePositions(): Called at the start of the game or upon reset. Forces a respawn() for every entity in the game to ensure a fresh, randomized distribution.
- resetGame(): A wrapper method that calls initializeGame() to restart the simulation without closing the application.
- getWinnerInfo(): Iterates through the animals list to find the highest score. Constructs and returns a string listing the winner(s) and their points.

1.2. Class: Animal (Abstract Base Entity) This abstract class serves as the root of the entity hierarchy, encapsulating shared attributes and defining the behavioral contract for all living creatures (Predator, Prey, ApexPredator). It implements Serializable to facilitate the save/load operations.

- Animal(String name, int x, int y, Type type, Era era): The constructor initializes the fundamental state of the entity. It sets the coordinate (x, y), assigns the biological type and era, and initializes score and abilityCoolDown to zero.
- moveTowards(int targetX, int targetY, Simulation model): A helper method implementing a Greedy Pathfinding Logic. Algorithm: It evaluates the 4 cardinal directions (Up, Down, Left, Right). Heuristic: For each neighbor, it calculates the Manhattan Distance (|x1-x2| + |y1-y2|) to the target coordinates. Validation: It checks boundaries (x >= 0, x < boardSize) and verifies that the cell does not contain an obstacle (!model.hasObstacle). Execution: It selects the neighbor that offers the minimum distance to the target and updates the animal's position using setPosition().
- randomMove(Simulation model): A fallback movement mechanism used when no clear target exists or paths are blocked.

- It iterates through possible directions in a randomized order or selects a random valid neighbor.
- It ensures the destination is within grid boundaries and obstacle-free before moving.

- reduceCooldown(): Manages the temporal constraints of special abilities.

- Checks if abilityCoolDown > 0.
- Decrements the counter by 1. This is called at the beginning of each turn to recharge abilities (e.g., Dash).

- addScore(int points) / loseScore(int amount): Encapsulation methods for state modification. They safely update the score attribute, ensuring data integrity during collision events.
- setPosition(int x, int y): Updates the entity's coordinates. This method is the single point of truth for location changes, making it easy to track movement bugs.
- abstract move(Animal target, Food food, Simulation model): Polymorphic Behavior. This abstract method forces every subclass to implement its own AI logic.

- Prey implements fleeing logic.
- ApexPredator implements hunting logic.

- Predator (Player) implements input-based logic (or empty logic if controlled via UI).

- abstract getMoves(int boardSize, Simulation model): Rule Enforcement. This abstract method requires subclasses to define their valid movement patterns (e.g., 1-tile walk vs. 3-tile jump) for UI highlighting and validation.
- abstract copy(): Prototype Pattern. Allows creating a deep copy of the animal instance. This is particularly useful for simulation lookaheads or state preservation.
- Getters (getX, getY, getName, getType, getEra, ...): Standard accessor methods providing read-only access to the entity's private fields, adhering to the principle of Encapsulation.

1.3. Class: ApexPredator (The Hunter AI) This concrete subclass extends Animal and implements a hostile AI logic. It represents the top of the food chain, actively hunting both the Player (Predator) and the Prey.

- ApexPredator(String name, int x, int y, Era era): Constructor. Calls the super constructor (Animal) with Type.APEX_PREDATOR, initializing its state.
- move(Animal target, Food food, Simulation model): The Decision Core. This is the entry point for the AI's turn. Step 1: Calls determineBestTarget(model) to decide who to hunt. Step 2: If no valid target is found (e.g., map is empty), it calls moveRandomly() to patrol. Step 3: If a target exists, it delegates the pathfinding to bestMove(priorityTarget, model).
- determineBestTarget(Simulation model): Priority System Logic. Primary Objective: It first checks for the Player (Predator). Since the Apex is designed to challenge the user, it targets the player regardless of distance, provided the player exists. Secondary Objective: If the player is unreachable or dead, it scans the animals list to find the nearest Prey. It calculates the distance to every Prey and selects the closest one (minDistance).
- bestMove(Animal target, Simulation model): Greedy Pathfinding Algorithm. Candidate Generation: It gathers all possible moves. Always includes standard walking moves (getWalkingMoves). If abilityCoolDown is 0, it includes special moves based on Era (e.g., straight-line charge for Past, radius jump for Future). Heuristic Evaluation: For each candidate coordinate (px, py), it calculates the Euclidean/Manhattan Distance to the

target's current position. Selection: It chooses the candidate that minimizes this distance. Cooldown Management: If a special move was chosen (distance > 1), it sets the cooldown (2 or 3 rounds depending on Era).

- manualCooldownTick(boolean abilityUsedJustNow): State Maintenance. This helper method manages the recharge cycle of special abilities. It is specifically designed to handle turns where the AI moves normally (walking) or stays idle. If a special ability was NOT used in the current turn (!abilityUsedJustNow) and the cooldown counter is greater than zero, it decrements the counter by 1. This ensures that abilities regenerate over time when not in use.
- getMovesByOffset(Simulation model, int dist, boolean straightLineOnly): Vector-Based Movement Engine. This private helper generates moves based on direction vectors. Raycasting: It implements a collision detection loop. For any move with distance > 1, it checks the intermediate cells (k = 1 to dist). If an obstacle is detected along the path, the move is discarded. This prevents the animal from phasing through walls during a charge. Directionality: If straightLineOnly is true, it restricts movement to cardinal directions (Up, Down, Left, Right), simulating a linear sprint.
- getRadiusMoves(Simulation model, int radius): Area-of-Effect Movement. Used specifically for the Future Era. It iterates through a square grid defined by the radius (dx, dy from -radius to +radius). This allows the Apex Predator to move to any valid cell within range, simulating a high-tech teleport or jump that ignores line-of-sight blocking (though the destination must still be valid).
- getWalkingMoves(Simulation model): Generates standard 1-tile movements in 8 directions (including diagonals), validating each via isValidMove().
- getStraightMoves(Simulation model, int distance) / getRadiusMoves(...): Era-Specific Movement Logic. Past/Present: Generates moves in straight lines (Cardinal directions), checking for obstacles along the path (Raycasting-like check). Future: Generates moves in a wider radius, simulating advanced mobility or teleportation.
- moveRandomly(Simulation model): A fallback behavior used when the AI is idle. It selects a random valid neighbor to keep the entity dynamic.
- isValidMove(int x, int y, Simulation model): Validation Layer. Boundaries: Checks if x and y are within [0, boardSize). Obstacles: Returns false if model.hasObstacle(x, y) is

true. Territoriality: Returns false if the target cell is occupied by another Apex Predator, preventing them from stacking or getting stuck on each other.

- getDistance(int x1, int y1, int x2, int y2): A helper method calculating the Manhattan Distance (|dx| + |dy|) used for heuristic scoring.
- getMoves(int boardSize, Simulation model): Overrides the abstract method to return all currently valid moves (Walking + Special if available) for UI rendering purposes.
- copy(): Creates a deep copy of the Apex Predator instance, preserving its current score and cooldown state.

1.4. Class: Food (Resource Entity) This class represents the passive resource entities within the simulation. It serves as a lightweight data container holding the spatial coordinates and identity of food items (e.g., Grass, Ferns). It implements the Serializable interface to ensure that food locations and types are accurately preserved and restored during the save/load process.

- Food(String name, int x, int y): Constructor. Initializes the food object with a specific name (which varies based on the Era) and its initial grid coordinates.
- setPosition(int newX, int newY): State Mutator. Updates the food's location on the grid. This method is primarily invoked by the Simulation.respawn() logic to relocate the food item to a new random coordinate immediately after it has been consumed by a Prey.
- getName(), getX(), getY(): Accessor Methods. These provide read-only access to the object's private fields, adhering to the principle of Encapsulation. They are frequently called by the Panel class to determine where to draw the food sprite and by the Simulation class to check for coordinate overlaps (collisions) with animals.

1.5. Class: Obstacle (Static Environment Entity) This class represents the immutable physical barriers scattered across the simulation grid. It acts as a spatial constraint for all moving entities (Animals), influencing pathfinding algorithms and movement validation logic. It implements the Serializable interface to ensure that the exact layout of the map is preserved across game sessions.

- Obstacle(int x, int y): Constructor. Instantiates a barrier at the specified grid coordinates. These coordinates are typically generated randomly by the Simulation class during the initialization phase to create unique map layouts.
- getX(), getY(): Accessor Methods. These provide read-only access to the obstacle's position. They are critical for the Simulation.isCellFull() and Animal.isValidMove() methods, which query these coordinates to prevent entities from walking through or spawning on top of rocks/trees.

1.6. Class: Predator (The Player Entity) This concrete subclass extends Animal and acts as the avatar for the user. Unlike AI agents, its behavior is not driven by an automated loop but is reactive to user input via the GUI. It encapsulates complex movement logic that varies significantly depending on the selected Era, providing distinct gameplay mechanics for Past, Present, and Future scenarios.

- Predator(String name, int x, int y, Era era): Constructor. Calls the super constructor with Type.PREDATOR. It initializes the score to zero, preparing the entity for a new game session.
- getMoves(int boardSize, Simulation model): Move Calculation Engine. This method is called by the Panel class to determine which grid cells should be highlighted for the player. Return Type: It returns an ArrayList<int[]> where each integer array represents a coordinate tuple {x, y, type}. The 'type' index indicates if the move is standard (0) or special (1). Logic:

1. Standard Moves: It always calculates immediate neighbors (1-tile radius) as valid walking options.
2. Special Abilities: If abilityCoolDown is 0, it calculates extended moves based on the Era:
    - Past Era: Calls addMoves with a range of 2 restricted to straight lines (Linear Dash).
    - Future Era: Calls addMoves with a range of 2 in all 8 directions (Omnidirectional Dash).

    - Present Era: Checks isApexClose(). If an Apex Predator is nearby, it triggers the addTwoMoves() logic (Adrenaline Rush).

- addTwoMoves(ArrayList<int[]> moves, Simulation model): Complex Pathfinding Helper (Present Era). This method simulates a "Walk + Walk" mechanic, allowing the player to move 2 tiles in non-linear patterns (e.g., L-shapes). Algorithm: It iterates through all valid first steps. For each valid first step, it iterates through all possible second steps. This ensures that the player cannot "jump" over obstacles but must have a clear path of connected tiles to reach the destination.
- addMoves(ArrayList<int[]> moves, Simulation model, int range, int isSpecial, boolean straightLineOnly): Generic Movement Generator. It calculates potential destinations based on the specified range and directionality. Path Checking: It uses a loop to check intermediate cells. If an obstacle exists between the current position and the target (raycasting), the path is considered blocked, and the move is not added. This prevents the player from phasing through walls.
- isCorrectMove(int x, int y, Simulation model): Validation Layer. Boundaries: Verifies that the coordinate is within the grid limits. Collisions: Checks if the cell contains an Obstacle or another Predator/Apex. Note that it allows moving onto Prey cells (since that constitutes an "Eat" action).
- copy(): Prototype Pattern Implementation. Returns a new Predator object with identical state (score, cooldown, position). This is essential for simulation lookaheads or saving state snapshots without reference issues.

1.7. Class: Prey (The Survivor AI) This concrete subclass extends Animal and implements a defensive AI logic. Its primary goal is to survive by avoiding Predators and Apex Predators while seeking Food to increase its score. It features a sophisticated scoring system to balance these conflicting needs based on the immediate danger level.

- Prey(String name, int x, int y, Era era): Constructor. Calls the super constructor with Type.PREY. It initializes specific flags like eatingDisabled, which is used for the Future Era's trade-off mechanics.

- move(Animal target, Food food, Simulation model): The Behavioral Tree. This is the main decision-making method. Step 1: Threat Assessment. It calls getNearestThreat(model) to scan the entire board for the closest predator. Step 2: Idle State. If no threat is nearby, it defaults to seeking food (if available) or moving randomly to explore the map. Step 3: Flee State. If a threat is detected within a dangerous radius, it delegates the logic to bestMove() to calculate the optimal escape route.
- moveTo(int targetX, int targetY, Simulation model): Targeted Movement Helper. This private method allows the Prey to navigate towards a specific destination (typically a Food source) when no immediate threats are present. Algorithm: It iterates through all valid immediate neighbors (1-tile radius). For each neighbor, it calculates the distance to the target coordinates. Selection: It adopts a greedy approach, immediately updating the Prey's position to the neighbor that minimizes the remaining distance to the target. This ensures the most direct path is taken.
- moveRandom(Simulation model): Idle State Behavior. A fallback mechanism used when the Prey detects no threats and no food sources. Algorithm: It first calls getValidNeighbors() to retrieve a list of all accessible adjacent cells (checking boundaries and obstacles). If valid options exist, it employs a SecureRandom instance to select one coordinate tuple uniformly at random and updates the entity's position. Purpose: This ensures the entity remains dynamic and unpredictable even when lacking specific environmental stimuli, preventing it from remaining static on the board.

- isValidMove(int x, int y, Simulation model): Validation Layer. This private helper acts as the final gatekeeper for all movement calculations. It performs a three-step verification for any target coordinate (x, y):

1. Boundary Check: Ensures the coordinate is strictly within the grid limits [0, boardSize).
2. Physical Constraints: Verifies that no Obstacle exists at the location using model.hasObstacle().
3. Collision Prevention: Checks if the cell is already occupied by another animal via model.getAnimalAt(). Unlike Predators (who can move onto Prey to eat), Prey entities treat all other animals as solid obstacles to avoid stacking or invalid overlaps.

- bestMove(Animal threat, Food food, Simulation model): Heuristic Evaluation Engine. This method determines the best coordinate to move to by assigning a "Survival Score" to every possible move. Candidate Generation: It gathers all valid neighbor cells. If abilityCoolDown is 0, it adds special escape moves:

- Past Era: L-Shaped Moves (similar to a Knight in Chess) to break line-of-sight.
- Present Era: 2-tile Hops (Sprint) to create distance quickly.
- Future Era: 3-tile Teleportation. Scoring: It iterates through each candidate and calls calculateScore(). The move with the highest score is executed. If a special move is chosen, the cooldown is triggered.

- calculateScore(int[] p, Animal threat, Food food, boolean issafe): The Utility Function. It assigns a numerical value to a potential move based on weighted factors:

1. Threat Distance (Weight: High): Maximizes the distance from the Predator.
2. Food Distance (Weight: Moderate): Minimizes the distance to Food.
3. Penalties: If the move is a Future Era teleport (hop), it applies a penalty because this move disables eating for the turn. Logic: Score = (DistanceToThreat * 10) - (DistanceToFood * 2). This ensures that safety is always prioritized over eating when danger is imminent.

- getNearestThreat(Simulation model): Sensing Mechanism. Iterates through the entire animals list to find the closest entity of type PREDATOR or APEX_PREDATOR using Manhattan distance calculations.
- getLMoves(Simulation model): Past Era Logic. Generates moves in an "L" pattern (2 squares in one cardinal direction, 1 square perpendicular), allowing the Prey to maneuver around obstacles and predators unpredictably.
- getMovesOffset(Simulation model, int dist): Generalized Movement Helper. This private method acts as the core engine for all distance-based movements. Instead of writing separate logic for 2-tile and 3-tile jumps, this method accepts a distance parameter to calculate coordinates in 8 directions. Crucially, it implements a "raycasting" loop (for (int k = 1; k < dist; k++)) to verify that the trajectory is clear of obstacles, preventing the Prey from phasing through solid barriers like rocks or trees during its jump.

- move(Animal target, Food food, Simulation model): Behavioral Decision Tree. This overridden method acts as the central intelligence node for the entity, determining its high-level strategy for the current turn. Logic Flow:

1. Threat Scanning: It first invokes getNearestThreat() to scan the environment for any immediate dangers.
2. Safe State: If no threats are detected, it prioritizes sustenance. It directs the entity to move towards the nearest Food source via moveTo(). If no food is available, it defaults to exploration behavior using moveRandom().
3. Danger State: If a threat is identified, it overrides all other desires and delegates control to the bestMove() method to execute optimal evasive maneuvers.
4. Cooldown Management: Finally, it ensures the ability cooldown is decremented correctly via manualCooldownTick() if no special ability was triggered.

- getLMoves2(Simulation model) / getLMoves3(Simulation model): Mobility Logic. Calculates destination cells that are 2 or 3 units away. It checks bounds and obstacles but allows jumping over empty space, simulating speed or teleportation.
- canEat() / resetEatingStatus(): State Management. Used specifically for the Future Era. If a Prey uses a long-range teleport (3 tiles) to escape, eatingDisabled is set to true. This method prevents the Prey from consuming food in the same turn it teleported, introducing a strategic trade-off between mobility and scoring.
- copy(): Creates a deep copy of the Prey instance, ensuring that its internal state (score, cooldown, specific flags) is preserved during simulation snapshots.

2.Package: foodchain.util

2.1. Enum: GameMode (Configuration Constants) This enumeration acts as a central configuration hub for the game's different eras. Instead of hardcoding file paths and display names throughout the application, this enum encapsulates specific data associated with each game mode (Past, Present, Future), ensuring a clean separation of configuration data from logic.

- GameMode(String displayName, String filePath, Animal.Era era): Constructor. A private constructor used internally to initialize the enum constants (PAST, PRESENT, FUTURE). It assigns the user-friendly name for the GUI, the relative path to the configuration text file, and the corresponding Era type used by the logic layer.
- getFilePath(): Resource Locator. Returns the relative string path to the configuration file (e.g., src/foodchain/past_animals.txt). This method is utilized by the Reader class to dynamically load the correct entity hierarchy based on the user's selection in the StartScreen.
- getEra(): Type Accessor. Returns the Animal.Era constant associated with the mode. This is used by the Simulation and Panel classes to determine logic rules (like movement types) and visual assets (background images) specific to the selected time period.
- toString(): UI Representation Override. Overrides the default toString() behavior to return the displayName (e.g., "Past (Dinosaur Era)") instead of the enum constant's name. This allows the enum objects to be added directly to Swing components like JComboBox, where they render as readable text options for the user while retaining their underlying object functionality.

2.2. Class: Log (System Diagnostics) This static utility class provides a centralized logging mechanism to record the runtime history of the simulation. It aids in debugging and allows users to review the sequence of events (movements, scores, game states) after the application concludes. It abstracts the low-level file writing operations from the main game logic.

- writeToFile(String message): Persistence Engine. This private helper method handles the actual writing to the disk. Stream Management: It initializes a BufferedWriter wrapped around a FileWriter. Crucially, the FileWriter is instantiated with the append flag set to true, ensuring that new log entries are added to the end of log.txt without overwriting existing data. Timestamping: It captures the current system time using LocalDateTime.now() and formats it into a readable HH:mm:ss string. This timestamp is prepended to every log entry, providing a precise chronological context. Resource Safety:

The method utilizes the try-with-resources statement. This ensures that the file stream is automatically closed after execution, preventing resource leaks and file locking issues.

- logSetup(String era, int totalRounds, int boardSize): Session Initialization. Records the configuration parameters at the start of a new game. It writes visual separators and details like the selected Era, Total Rounds, and Grid Size, making it easy to distinguish between different sessions within the same log file.
- logMove(String actorName, int startX, int startY, int endX, int endY): Spatial Tracking. Formats movement data into a standardized string [MOVE]. It records the entity's name along with its starting and ending coordinates, allowing for a step-by-step reconstruction of the game flow.
- logScore(String actorName, int points, String reason): State Change Tracking. Records updates to an entity's score. It evaluates the points parameter to determine if the action was a gain (GAINED) or a loss (LOST). It also includes a reason string (e.g., "Ate Food"), providing context for the score change.
- logStart(int round) / logEnd(int round): Flow Control. Logs the beginning and termination of each round. This structure divides the log file into logical blocks, making it easier to analyze turn-based behavior.
- logEnd(int round): Round Finalization. Logs the termination of the current round and appends an empty line to create a visual break in the text file, improving readability.
- logWinner(String winnerInfo) / logGameOver(): Session Termination. Records the final outcome of the simulation, including the winner's name and score, followed by a closing footer.
- logGameOver(): Session Termination. Writes a simple "GAME OVER" footer to denote the absolute end of the simulation logic.
- log(String msg): Generic Interface. A public wrapper allowing other classes to write arbitrary debug messages to the log file without worrying about formatting or file handling.

2.3. Class: Reader (File Parser) This static utility class is responsible for determining the initial biological hierarchy of the simulation by parsing external configuration files. It acts as a factory that translates raw text data into live objects within the game engine.

- loadFile(Simulation model, String filePath, Era era): Data Injection Engine. This method orchestrates the entire setup process for a new game session. Step 1: File Access. It initializes a BufferedReader wrapped around a FileReader to access the target text file (e.g., past_animals.txt). It uses a try-with-resources statement to ensure the file stream is closed automatically, preventing memory leaks. Step 2: Content Filtering. It reads the file line-by-line using a while loop. It specifically looks for lines starting with the keyword "Food Chain", ignoring comments or empty lines. Valid lines are stored in a temporary list. Step 3: Stochastic Selection. It utilizes the java.security.SecureRandom class to select one configuration string from the list. This ensures that the game setup is unpredictable and varies between sessions. Step 4: Parsing & Instantiation. The selected string (e.g., "Food Chain 1: Lion, Cheetah, Bunny, Grass") is split using the colon and comma delimiters. The method extracts the specific names for the Apex Predator, Predator, Prey, and Food. Step 5: Model Population. It first clears the current state of the Simulation model to remove old entities. Then, it instantiates the corresponding concrete classes (ApexPredator, Predator, Prey) with the parsed names and adds them to the model's tracking lists. Error Handling: The entire operation is guarded by a try-catch block catching IOExceptions. If a file is missing or unreadable, it prevents a crash by displaying a user-friendly error message via JOptionPane.

3.Package: foodchain.view

3.1. Class: Dashboard (Heads-Up Display) This class extends JPanel and serves as the primary information feedback loop for the user. Located on the right side of the game window, it visualizes the real-time state of the simulation model, distinct from the grid view. It employs Java 2D graphics to render stylized statistics cards for each active entity.

- Dashboard(Simulation model): Constructor. Initializes the panel's layout properties. It sets the preferred dimensions and background color to ensure visual consistency with the main frame. It stores the reference to the Simulation model to query data during painting cycles.
- paintComponent(Graphics g): The Rendering Pipeline. Overrides the standard Swing painting method. Graphics Configuration: Casts the Graphics object to Graphics2D to enable Anti-Aliasing (RenderingHints.KEY_ANTIALIASING). This ensures that text and rounded shapes appear smooth and professional. Layout Management: It dynamically calculates the vertical spacing between elements. First, it draws the global game state (Current Round), then iterates through the active animals to delegate the drawing of specific entity cards via drawCard().
- drawCard(Graphics2D g, int x, int y, int w, int h, Animal animal): UI Component Renderer. Draws a customized information block for a specific animal. Visual Coding: It selects a background color based on the animal's type (Red for Apex, Pink for Predator, Blue for Prey), providing immediate visual recognition. Dynamic Status Text: Beyond static data like Name and Score, this method visualizes the state of the entity's special ability. Context-Sensitive Logic: For the Predator in the Present Era, it queries model.isApexClose(). If true, it renders the ability text in red as ACTIVE! (RUN!), informing the player that their passive speed boost is triggered. For other eras/animals, it displays the numeric Cooldown value or a Ready status.
- drawRoleCard(Graphics2D g, int x, int y, int w, int h, Color bg, String title, Animal animal): Primary UI Renderer. This is the main implementation that handles the detailed visualization of an animal's status. Visual Coding: It renders the card background using the specific color associated with the animal's type (Red, Pink, or Blue). Data Binding: It extracts and displays the animal's Name and current Score. Context-Sensitive Logic: It performs conditional checks to display the correct ability status.
- drawCard(Graphics2D g, int x, int y, int w, int h, Color bg, String title): Generic Component Renderer. A helper method responsible for drawing the static "Game Information" box. It renders the background rectangle with a semi-transparent shadow offset, draws the border, and places the title text with a separator line.

- refresh(): Update Trigger. A public method called by the Panel class at the end of every cinematic turn. It invokes repaint(), forcing the Swing framework to redraw the component with the latest data from the model.

3.2. Class: Frame (Main Window) This class extends JFrame and serves as the top-level container for the application's graphical interface. It acts as the structural skeleton, organizing the layout of the primary view components and providing a standard menu bar for high-level game management controls.

- Frame(Simulation model): Constructor. Responsible for assembling the main application window. Window Properties: It sets the title dynamically based on the current Era of the player (e.g., "Food Chain Game - PRESENT") and configures the default close operation to exit the application. Layout Management: It utilizes a BorderLayout to divide the window into two distinct regions. The central area is occupied by the Panel (the game grid), while the eastern region houses the Dashboard (statistics). Menu Construction: It initializes a JMenuBar containing a "Game" menu. It programmatically adds three JMenuItem components: "New Game", "Save Game", and "Load Game". Event Handling: It attaches functional listeners (Lambda expressions) to these menu items.

- New Game: Disposes of the current window and instantiates a new StartScreen to reset the configuration.
- Save Game: Opens a JOptionPane input dialog to request a filename, then triggers model.saveGameToText().
- Load Game: Opens a JOptionPane to request a target file. It calls the static Simulation.loadGameFromText() method. If successful, it disposes of the current window and creates a new Frame with the loaded model, effectively refreshing the entire application state.

3.3. Class: Panel (Game Board Renderer) This class extends JPanel and acts as the primary view component. It is responsible for rendering the simulation grid, managing visual assets, handling user input via mouse/keyboard, and orchestrating the turn-based animation flow using multithreading.

- Panel(Simulation model, Dashboard infoPanel): Constructor. Initializes the panel's dimensions based on the SCREEN_SIZE constant. It sets the background color, instantiates the input listeners (MouseAdapter, KeyAdapter), and triggers the resource loading process via loadImages().
- loadImages(): Dynamic Resource Management. Identifies the current Era from the model and loads the corresponding theme assets (Backgrounds, Sprites for Apex/Predator/Prey/Food). It uses the helper method loadSmartImage() to scan multiple directory paths and file extensions, ensuring the application remains robust even if file formats vary (e.g., .png vs .jpg).
- loadSmartImage(String baseName): Asset Loading Helper. Iterates through multiple directory paths and file extensions (.png, .jpg, .jpeg) to find and load an image file. This robust approach prevents crashes caused by missing file extensions or slight naming variations.
- paintComponent(Graphics g): The Rendering Pipeline. Overrides the standard Swing painting method to draw the game state frame-by-frame. It utilizes Graphics2D for advanced drawing operations. The rendering order is strictly layered to ensure correct occlusion:

1. Background & Grid: Draws the era-specific wallpaper and semi-transparent grid lines.
2. Static Entities: Renders Obstacles and the list of Food items.
3. Highlights: Iterates through the Player's valid moves. It renders Green rectangles for standard steps and Yellow rectangles for special ability ranges.
4. Dynamic Entities: Draws the Animal sprites. It employs color-coded backups (Red/Blue/Orange rectangles) if image assets fail to load.
5. Visual FX Overlays: Renders temporary visual cues such as Death Markers (Red Crosses), Spawn Indicators (Cyan Circles), and Movement Trails (Colored Lines connecting start/end points).

- runCinematicTurn(): Concurrency & Animation Engine. This method solves the problem of UI freezing during complex calculations. It spawns a new Thread (Worker Thread) separate from the Swing Event Dispatch Thread (EDT). Sequence: It executes the game logic in a strict order: Player Move -> Delay -> Prey Moves -> Delay -> Apex Moves -> Delay -> End Round. Thread Safety: Since Swing components are not thread-safe, all UI updates (repaint calls) triggered from this thread are wrapped in SwingUtilities.invokeLater().
- draw(Graphics g): The Primary Rendering Engine. This public method encapsulates the core drawing logic, separating it from the Swing component lifecycle for better code organization. It executes a strict "Painter's Algorithm," drawing elements from back to front to establish the visual hierarchy (Z-order). Layer 1 (Environment): It first renders the era-specific background image scaled to the screen size. Then, it overlays a semi-transparent white grid to demarcate the 20x20 cell boundaries. Layer 2 (Static Objects): It iterates through the obstacles and food lists. Food items are rendered with a translucent yellow halo to improve visibility against complex backgrounds. Layer 3 (Player Hints): If the turn is idle (not animating), it retrieves the Player's valid moves and renders them as colored overlays—Green for standard walking tiles and Yellow for special ability ranges. Layer 4 (Agents): It iterates through the animal list to render sprites. It implements a fallback logic: if the image asset (imgApex, imgPredator, imgPrey) is null, it renders a colored rectangle (Red/Orange/Blue) instead. It also draws the animal's name above the sprite using a dual-layer text technique (Black shadow, Colored foreground) for readability. Layer 5 (FX Overlays): Finally, it draws the highest-priority visual effects: Red "X" markers for death locations, semi-transparent "Ghost Trails" connecting movement points, and Cyan circles indicating where new entities have spawned.
- handleVisuals(): Animation Synchronization. A helper method used within the animation thread. It pauses execution briefly using Thread.sleep() to allow transient visual effects (like the "SPAWN" text or Death markers) to remain on screen long enough for the user to perceive them before they are cleared.
- checkGameOver() / showModernGameOverScreen(boolean playerAlive): End-Game Logic. Checks if the win/loss conditions are met. Instead of a standard message box, it

constructs a custom JDialog. This dialog features a dynamic background image matching the winning animal and provides styled buttons to Restart or Exit the application.

- gameOverScreen(boolean playerAlive): Custom End-Game UI. This private method constructs a rich, modal JDialog to display the session results, replacing standard system message boxes. Dynamic Background Logic: It determines the visual theme based on the winner. If a single animal wins, it sanitizes the winner's name (e.g., "T-Rex" becomes "trex") and attempts to load a specific victory background image matching that animal and era. If it's a draw, it defaults to the generic Era background. UI Composition: It utilizes a semi-transparent JPanel to overlay text on the background image. It dynamically sets the title text ("YOU DIED!", "GAME OVER!", or "DRAW!") and color (Red, Green, or Orange) based on the player's survival status. Interactive Controls: It generates styled "Play Again" and "Exit" buttons. The restart logic handles a full simulation reset and reloads visual assets via reflection to ensure the next session starts cleanly.
- button(String text, Color bgColor): UI Factory Helper. This private method abstracts the creation of Swing buttons to enforce a consistent "Flat Design" aesthetic. Instead of repeating styling code, it generates a JButton with a standardized configuration: it disables the focus painting for a cleaner look, sets the font to "Segoe UI", applies internal padding via an EmptyBorder, and changes the mouse cursor to a Hand icon to improve UI affordance.
- MyMouseAdapter (Inner Class): Input Translation. Captures mouse clicks on the panel. It translates screen pixel coordinates (e.g., 450px) into grid indices (e.g., x=10, y=10). It validates if the clicked cell is a valid move for the Predator and, if so, updates the model and triggers the cinematic turn.
- MyKeyAdapter (Inner Class): Shortcut Management. Listens for key presses. It maps specific keys (e.g., 'S') to high-level system functions like saving the game state to a text file.

3.4. Class: StartScreen (Configuration Interface) This class extends JFrame and provides the initial user interface for simulation configuration. It allows users to define key parameters such as

grid dimensions, round limits, and the biological era before the simulation engine is initialized. It features custom-painted Swing components to establish a modern aesthetic.

- StartScreen(): Constructor. Initializes the application's entry window. Layout Strategy: It utilizes a GridBagLayout to center the configuration card dynamically, regardless of the window size. Visual Customization: Instead of a standard content pane, it sets a custom GradientPanel as the background. It also initializes the input components (comboboxes, spinners) and attaches them to the main panel.
- createLabel(String text): UI Factory Helper. Generates JLabel objects with pre-defined font (Segoe UI) and color properties, ensuring typography consistency across the menu.
- styleComboBox(JComboBox box) / styleSpinner(JSpinner spinner): Component Styling. These private methods apply specific background colors and fonts to standard Swing input components, ensuring they blend seamlessly with the custom dark theme of the card panel.
- styleSpinner(JSpinner spinner): UI Customization. This private helper method ensures visual consistency for the numerical input component. Unlike standard components, a JSpinner is a complex container; therefore, this method accesses its inner "Editor" component. It specifically retrieves the underlying text field to enforce a white background and black text, overriding the default look-and-feel to ensure readability against the dark card background.
- addInputFields(JPanel panel): Component Assembly. A helper method that constructs the form layout. It creates labeled input pairs (e.g., "Select Era" + JComboBox) and adds them to the central panel using GridBagConstraints for precise alignment and padding.
- startGame(): Validation & Initialization Logic. Triggered when the "Start Simulation" button is clicked. Input Validation: It retrieves the value from the roundSpinner. If the value is less than 10, it interrupts the process and displays a warning via JOptionPane, enforcing the minimum game duration rule. Model Instantiation: If inputs are valid, it creates a new Simulation instance with the selected Board Size, Round Count, and Era. Data Loading: It calls the static Reader.loadFile() method to populate the new simulation model with entities from the corresponding text file. Transition: Finally, it disposes of the StartScreen and launches the main Frame, passing the initialized model.

- loadGame(): State Restoration. Triggered by the "Load Saved Game" button. It prompts the user for a filename and calls Simulation.loadGameFromText(). If the deserialization is successful, it launches the main Frame with the restored game state; otherwise, it notifies the user of the failure.
- ModernButton (Inner Class): UI Customization. Extends JButton to replace the standard OS-style buttons with rounded, flat-design buttons. Event Handling: Implements a MouseListener to track hover events. It sets a boolean flag whenApproached to true when the mouse enters the button area, triggering a repaint() to lighten the background color (Hover Effect). Rendering: Overrides paintComponent using Graphics2D with Anti-Aliasing enabled. It draws a filled rounded rectangle (fillRoundRect) and centers the text label manually using FontMetrics.

References:

Oracle Java Documentation: Swing, AWT, and Concurrency.

Course Lecture Notes.

Java IO Documentation.